IACE PLATFORM · OPERATIONS

The deployment pipeline

How a commit on the staging branch becomes a running API and two published websites — every file, every step, and what to do when one fails.

Pipeline GitHub Actions → ECR → EC2 over SSM → S3 + CloudFront
Written 8 October 2026, after the first releases went out through it
Source of record .github/workflows/, deploy/, docs/04-infrastructure.md §14

Contents

1. The pipeline in one page 4
2. The words you need 6
3. Where everything runs 8
4. The seven workflow files 10
5. A staging release, minute by minute 17
6. How GitHub is allowed into AWS 18
7. The deploy folder, file by file 21
8. The environment file 27
9. When something fails 29
10. Everyday operations 31
11. Known gaps today 33
12. What production needs 34

The deployment pipeline

This document explains how a push to the `staging` branch of the IACE exam platform becomes a running staging environment: which files do the work, what each line in them does, the order things happen in, what to do when a step fails, and what is still missing before production. It describes the repository and the AWS and GitHub settings as they stood on 8 October 2026.

Read sections 1 to 3 first; they give the whole picture in a few pages. Sections 4 to 8 are reference, file by file and line by line. Sections 9 to 12 cover failures, everyday commands, the known gaps and what production still needs. Line numbers refer to the working tree on 8 October 2026, which includes commit 767aead. That commit is on the local `staging` branch and has not been pushed, so the copy on GitHub differs from these line numbers until it is.

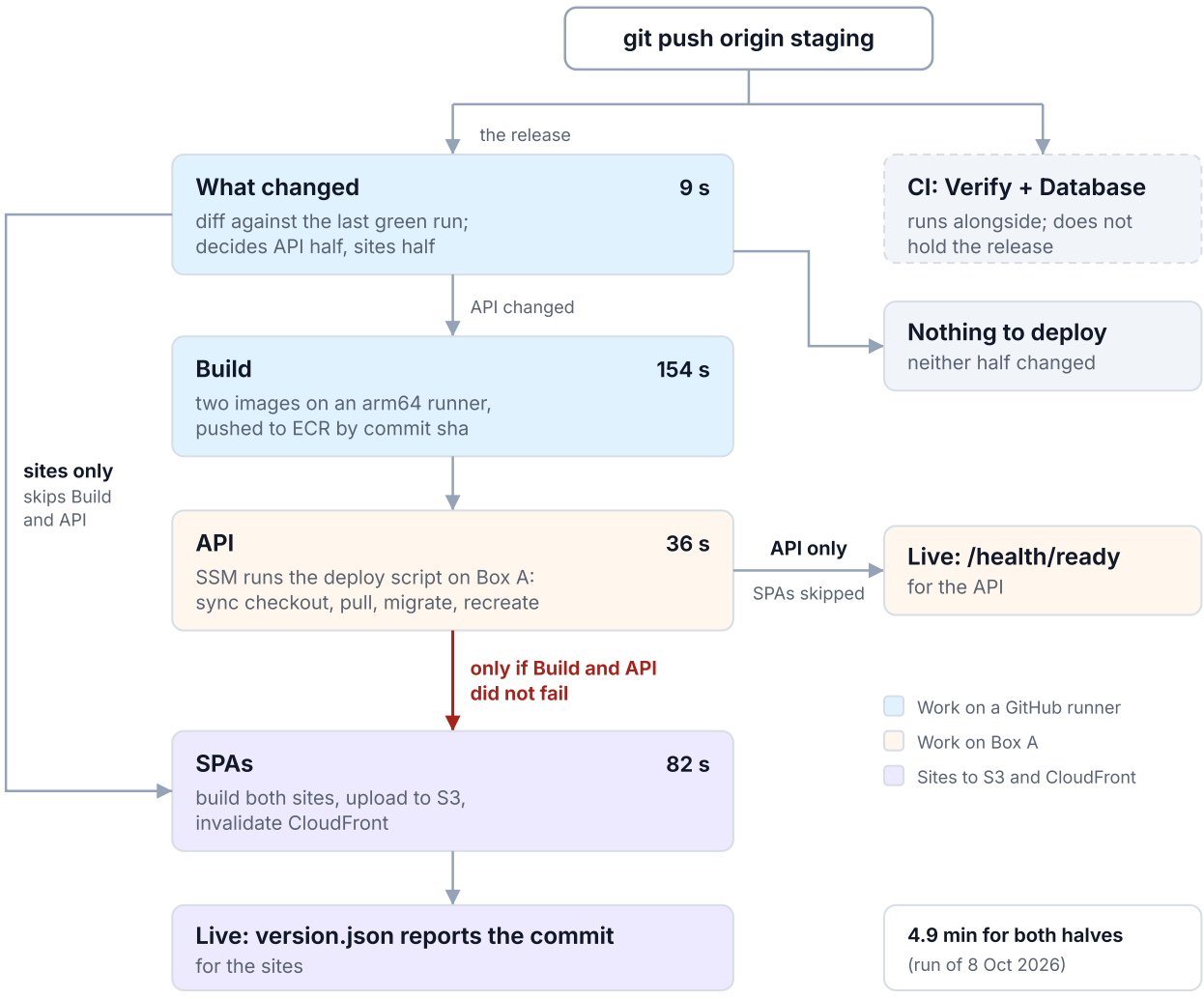
The workflow files in `.github/workflows/` and the files in `deploy/` win over this document: where they disagree, the files are right and this text is stale. `docs/04-infrastructure.md` §14 is the short form of the same story. The settings that live in AWS and GitHub rather than in the repository (the IAM role, the variables, the environments) are described as they were read on 8 October 2026; no workflow file shows them.

1. The pipeline in one page

A push to `staging` starts two workflow runs at the same moment. A workflow is a YAML file in `.github/workflows/`, and a run is one execution of it. The first run, **CI** (`ci.yml`), only checks the code: formatting, types, tests, a build, and every database migration applied to an empty database. The second, **Staging** (`staging.yml`), is the release. It does the following, in order.

1. **What changed** (9 seconds). Compares this push with the last Staging run that succeeded and decides whether the API half, the sites half, both or neither must go out.
2. **Build** (about 2.5 minutes), only when the API half changed. Builds the API as two Docker images on an ARM machine and pushes both to ECR, Amazon's store for container images, tagged with the commit's full sha (its 40-character id).
3. **API** (about 36 seconds), only when the API half changed. Signs in to AWS, finds Box A (the EC2 server that runs the API containers), and sends it one shell script through SSM Run Command. The script moves the box's copy of the repository to this commit, pulls the new images, runs the database migrations, replaces the API containers and deletes old images.
4. **SPAs** (single-page applications, about 80 seconds), only when the sites changed. Builds the student and admin sites, uploads them to S3 and tells CloudFront to forget the few files that must never be cached.
5. Every job writes a short report to the run's summary page, so a run that deployed nothing says so.

A PUSH TO STAGING, FROM COMMIT TO LIVE



The ordering rule: the API always goes before the sites. A browser tab that is already open keeps its old site code until the tab is closed, so the API must go on serving the previous client anyway. The reverse order is not safe: a new site can send a field that an older API strips, and the data in that field is lost. For the same reason the SPAs job does not run when the build or the API job failed or was cancelled. Publishing is withheld on purpose.

> **CI does not gate the staging deploy.** Both runs start on the same push and run side by side, so a push that fails its tests still deploys to staging. That is deliberate for an environment whose purpose is to be ahead of production. Section 11 lists it as gap 4.

2. The words you need

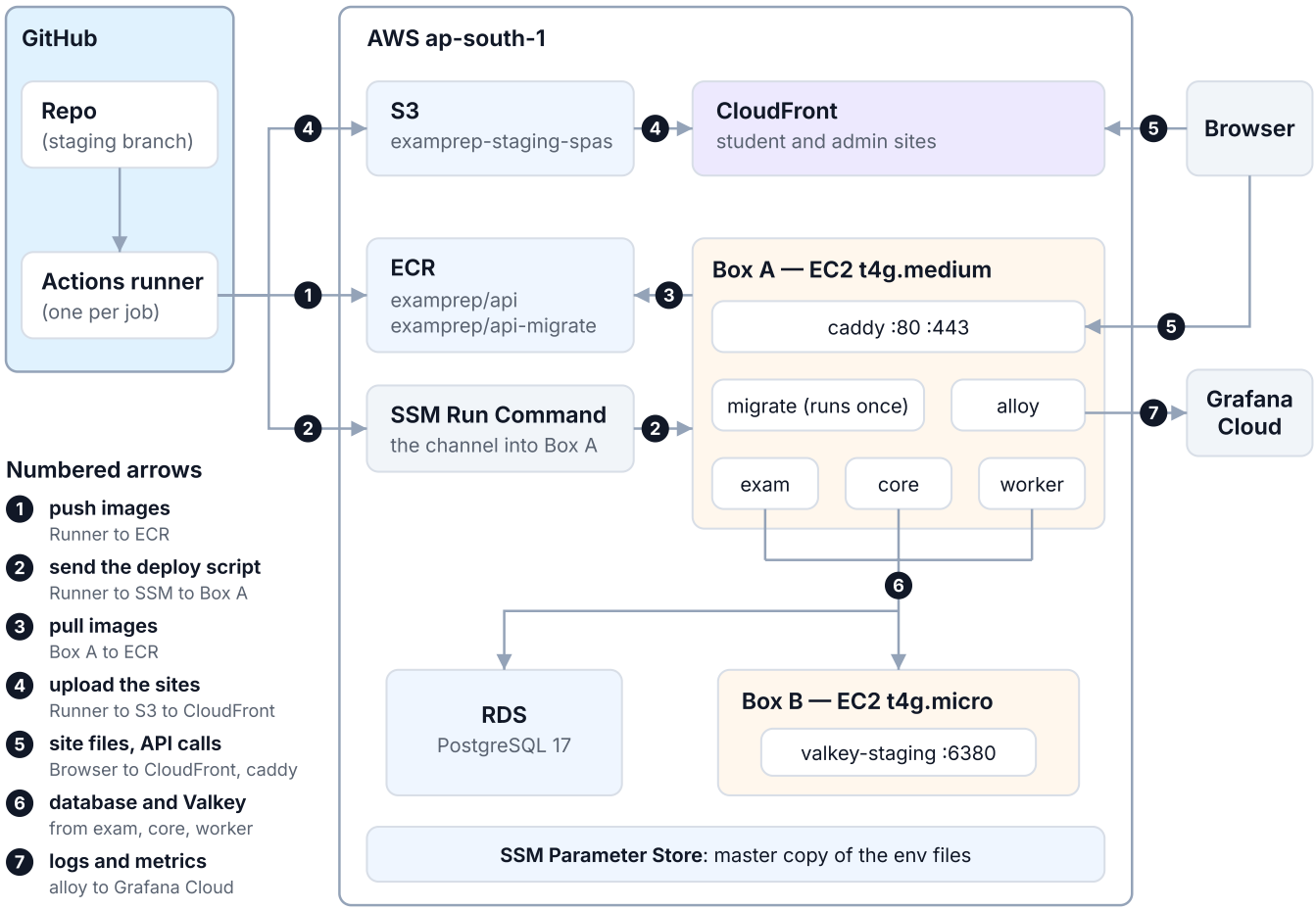
Each term is defined as it is used in this project.

Term	What it means here
CI	Continuous integration: checks that run automatically on every push. Here, `ci.yml`, which never deploys.
CD	Continuous delivery: carrying a change to a running environment automatically. Here, `staging.yml` and the files it calls.
Workflow	A YAML file in `.github/workflows/` telling GitHub what to run and when. This repository has seven.
Job	A group of steps run on one fresh machine. Jobs run in parallel unless one lists another under `needs`.
Step	One command (`run:`) or one ready-made action (`uses:`) inside a job.
Runner	The machine GitHub lends a job and destroys afterwards. `ubuntu-24.04-arm` is an ARM machine, used for image builds because both boxes are ARM (Graviton).
Trigger	The `on:` block of a workflow: a push, a pull request, a schedule, a manual button, or another workflow's call.
Reusable workflow	A workflow that others call, declared with `workflow_call`. A leading underscore in the file name marks it as not startable on its own.
Environment (GitHub's)	A named bundle in the repository settings, here `staging` and `production`, holding variables and optional protection rules. A job that declares one receives its variables and an identity AWS accepts.
Variable	A named, visible value in GitHub settings, read as `vars.SPA_BUCKET`. This pipeline uses no secrets.
Image	A packaged copy of the API and everything it needs to run, built from `apps/api/Dockerfile`. A container is an image that is running.
Registry (ECR)	Amazon Elastic Container Registry, the store images are pushed to and pulled from. Repositories: `examprep/api` and `examprep/api-migrate`.
Tag	A label on an image. The sha tag is the commit's full id: it never moves, and a rollback names it. The moving tag (`staging`, or `prod` on main) is re-pointed at every build. Tags in ECR are mutable.
OIDC	OpenID Connect. How a job proves who it is to AWS with no stored password: GitHub signs a short-lived token and AWS trades it for temporary credentials (section 6).
IAM role	An AWS identity with a list of permitted actions. Jobs assume `examprep-github-actions`; the boxes use the instance profile `examprep-box`.
SSM Run Command	An AWS feature that runs a shell script, as root, on an instance through an agent that dials out. There is no SSH, no key and no inbound port.
Docker Compose	The tool that reads `deploy/compose.yml` and starts the containers together.

Term	What it means here
CloudFront invalidation	A request that CloudFront, AWS's content delivery network, drop its cached copy of named files, so the next request fetches them from S3.
Service worker	A script (`sw.js`) the student site installs in the browser. A new one takes over only when every tab of the old one is closed, so "deployed" and "students see it" are different moments.

3. Where everything runs

WHERE EACH PIECE RUNS, AND WHO TALKS TO WHOM



The repository is checked out in two places: on a GitHub runner for every job, and on Box A at `/opt/examprep`. Box A reads `compose.yml`, the Caddyfile and the Alloy configuration from its own checkout, which is why the deploy script moves that checkout first. The API images come from ECR. The two sites never touch the boxes: they go from the runner straight to S3.

Machine or service	What runs on it	File that governs it
GitHub runners	Every job; the image build uses the ARM runner.	`.github/workflows/*.yml`
GitHub Actions cache	Layers from earlier image builds.	`build-api.yml` lines 78–79, 92–93
ECR	Repositories `examprep/api` and `examprep/api-migrate`; a lifecycle rule keeps the last 10 images.	`build-api.yml`, `apps/api/Dockerfile`
Box A: EC2 `examprep-api-staging`, t4g.medium, Elastic IP	The checkout; Caddy, the exam, core and worker containers, the one-shot migrate container, Alloy.	`deploy/compose.yml`, `Caddyfile`, `alloy/config.alloy`, `prune-disk.sh`, `deploy/.env` (not in git)

Machine or service	What runs on it	File that governs it
Box B: EC2 `examprep-valkey`, t4g.micro	Valkey, a Redis-compatible store: staging on port 6380, production on 6379 (defined, not started).	`deploy/valkey/compose.yml`, `deploy/valkey/.env` (not in git)
RDS `examprep`	PostgreSQL 17.11 on db.t4g.micro, single-AZ, 7-day backups.	`prisma/migrations`, applied by the migrate container
S3 `examprep-staging-spas`	Both sites, under the prefixes `student/` and `admin/`.	`deploy/publish-spas.sh`
S3 `examprep-staging-media`	Question images.	The API; the pipeline does not touch it.
CloudFront	`staging.examprep.iace.co.in` (student), `admin.staging.examprep.iace.co.in` (admin), `media.staging.examprep.iace.co.in` (images).	Invalidations in `deploy/publish-spas.sh`
SSM	Run Command (the deploy) and Parameter Store (`/examprep/staging/env`, `/examprep/valkey/env`, the master copies of the env files).	`_deploy-api.yml`, `deploy/README.md`
IAM and the OIDC provider	The deploy role and the instance profile.	AWS console only
Grafana Cloud	Loki (logs) and Prometheus (metrics), fed by Alloy.	`deploy/alloy/config.alloy`
DNS and Let's Encrypt	`api.staging.examprep.iace.co.in` points at Box A's Elastic IP; Caddy fetches its certificate.	`deploy/Caddyfile`; the DNS record is outside the repository

4. The seven workflow files

File	Runs when	Role
`ci.yml`	Push to `staging` or `main`, every pull request, a manual run, Monday 03:00 UTC	Checks only. Never deploys.
`staging.yml`	Push to `staging`, or a manual run with an image tag	The release: what changed, build, API, SPAs.
`build-api.yml`	Called by `staging.yml`; push to `main` touching API paths; manual	Builds two images and pushes them to ECR. Never deploys.
`_deploy-api.yml`	Called by `staging.yml` and `production.yml`	Runs the deploy script on Box A through SSM.
`_publish-spas.yml`	Called by `staging.yml` and `production.yml`	Builds and uploads the two sites.
`production.yml`	Nightly at 20:30 UTC (02:00 IST), or manual	Disabled in GitHub's Actions settings since 7 October 2026.
`scale.yml`	Manual only	Changes container counts and optionally the instance size.

Two settings recur. `permissions: id-token: write` lets a job ask GitHub for the OIDC token, and `environment:` on a job is what makes AWS accept that token (section 6). `concurrency:` limits how many runs of a group may be active at once.

`ci.yml`

Runs beside every release and on every pull request. Two jobs run in parallel: **Verify** (format, dependency alignment, lint, typecheck, test, build; 6.7 minutes) and **Database** (apply every migration to an empty Postgres, check the migrations match the schema, run the SQL tests; 7.8 minutes). A third job, **Docs**, runs only on the weekly cron or a manual run. Nothing in it deploys.

Lines	What it does
3–11	Triggers: a push to `staging` or `main`, any pull request, a manual run, and a cron at 03:00 UTC every Monday.
15–20	One run per branch at a time: a newer push cancels the older run, except on `main`. The token may only read the repository.
22–27	A placeholder `DATABASE_URL` that Prisma needs but never connects to, and a switch that stops Corepack asking a question nobody can answer.
33–40	Job Verify. Its `if` skips the Monday run, which exists for the Docs job alone. 15-minute limit.
43–62	Check out, install the pnpm version pinned in `package.json`, install Node from `.nvmrc`, run `pnpm install --frozen-lockfile` (fails instead of resolving a new version), generate the Prisma client.
67–91	Format, dependency alignment, lint, typecheck, test, build. Each has `if: '!cancelled()'`, so each still runs after an earlier one failed and one push reports every problem.
94–116	Job Database: same schedule guard, 10-minute limit, a throwaway Postgres 17 container with a health check.

Lines	What it does
119–161	The same checkout, pnpm, Node and install steps; create the shadow database; apply every migration to the empty database as a deploy does; `pnpm db:check` fails if `schema.prisma` changed without a migration; create the test database, build `contracts`, run `pnpm test:db`.
169–187	Job Docs: only on the schedule or a manual run; `pnpm docs:check` compares the docs with the schema and gates nothing.

`staging.yml`

The release. It runs on every push to `staging`, and on a manual **Run workflow** click that supplies an image tag. A manual run treats both halves as changed, builds nothing and deploys the tag you typed.

Lines	What it does
5–13	Triggers: a push to `staging`, or a manual run with an `image_tag` input (required, default `staging`, the last image built).
15–17	`id-token: write` lets jobs request the OIDC token; `contents: read` is the least a checkout needs.
19–21	Group `staging-release`: one release at a time. `cancel-in-progress: false` makes a new run wait instead of killing the one in progress.
26–34	Job What changed. `actions: read` lets `gh run list` read run history. It publishes two outputs, `api` and `spa`.
38–39	Checks out the whole history (`fetch-depth: 0`); the range below reaches back to the last green run, which a shallow clone cannot see.
42–46	`GH_TOKEN` gives the `gh` command access. `both()` sets `api` and `spa` to true.
49	Anything but a push (a manual run) has no range to inspect, so both halves deploy.
54–55	Asks GitHub for the commit of the newest successful Staging run on `staging`. A failed lookup is tolerated, not fatal.
58	No green run yet, a failed lookup, or a sha this clone lacks (after a force-push): deploy both rather than guess.
62	Lists the files changed between that commit and this one, with two dots so a force-push whose old head is not an ancestor still compares the two trees.
69–70	Sets `api` and `spa` from the file list with the two patterns below.
76–85	Writes the summary: both shas, the file count, the two flags and, when both are false, **Nothing to deploy.**
90–97	Job Build calls `build-api.yml`. Its `if` (line 93): only on a push, and only when `api` is true. A manual run builds nothing, which keeps a rollback a tag.
101–112	Job API calls `_deploy-api.yml` for `staging`, with the typed tag on a manual run and this commit's sha on a push.
117–126	Job SPAs calls `_publish-spas.yml` for `staging`.

What the two patterns match (lines 69 and 70):

```
api:  ^(apps/api/|packages/(contracts|config)/|prisma/|pnpm-
lock\.yaml$|package\.json$|deploy/(compose\.yml|Caddyfile|prune-disk\.sh|alloy/))
spa:  ^(apps/exams|apps/admin|packages|deploy/publish-spas\.sh)/?
```

The API half is true when anything changed under `apps/api/`, `packages/contracts/`, `packages/config/` or `prisma/`, when the lockfile or the root `package.json` changed, or when one of the four deploy files the box reads changed. Only `contracts` and `config` count because they are the only workspace packages the API imports; a screen change therefore does not rebuild an image or restart the API. The sites half is true when anything changed under `apps/exams`, `apps/admin` or `packages/`, or when `deploy/publish-spas.sh` changed. The lockfile and root `package.json` count for the API half only. A change to docs, a workflow file or the mobile app matches neither.

The `if` on the API job (line 104). `always()` makes the job run even when Build was skipped, as on a manual run. It then requires the `api` flag to be true and Build not to have failed or been cancelled. **The `if` on the SPAs job (line 120)** also starts with `always()`, requires the `spa` flag to be true, and requires that none of What changed, Build and API failed or was cancelled (`contains(needs.*.result, ...)` checks all of them). A job that was merely skipped does not block it, so a sites-only push still publishes.

Why the diff is measured from the last green run. The obvious range, this push against the previous push, is wrong. Runs queue behind one another, and GitHub keeps only one pending run per concurrency group, cancelling older pending ones. A run that failed or was cancelled deployed nothing, so its changes must still count in the next run. Suppose push A changes the API and its build fails, and push B changes only a screen. Measured against A, B sees no API change, publishes a new bundle and leaves the old API running under it. Measured from the last green run, B's range includes A's files and the API half is true.

`build-api.yml`

Builds the two API images and pushes them to ECR. It never deploys: `_deploy-api.yml` decides when an image reaches a box, which makes a rollback a tag instead of a rebuild.

Lines	What it does
8–19	Triggers: called by another workflow; a push to `main` touching `apps/api/`, `packages/`, `prisma/`, the lockfile, root `package.json` or this file; a manual run.
25–29	One build per branch at a time (`build-api-staging`, `build-api-main`), so a queued `main` build cannot cancel the build a staging release is waiting on.
31–32	The registry address, shared by the steps.
35–42	Job Build and push. Native ARM runner, which has no minutes limit only while the repository is public (section 12). 45-minute limit. The environment is `production` on `main` and `staging` otherwise, because AWS checks it.
45–51	Checks out the commit, then chooses the moving tag: `prod` on `main`, `staging` otherwise.
53–58	Trades the OIDC token for temporary AWS credentials on the role in `vars.AWS_ROLE_ARN`, then signs Docker in to ECR.

Lines	What it does
60–63	Sets up buildx, Docker's builder. `cleanup: false` skips the end-of-job teardown, about 26 seconds, on a runner that is thrown away anyway (added in commit 767aead).
67–79	Builds the `runtime` target for ARM and pushes `examprep/api` with two tags, the sha and the moving tag. Layers are read from and written to the GitHub Actions cache (`type=gha`; `mode=max` stores every layer).
81–93	The same for the `migrate` target, pushed as `examprep/api-migrate`.
95–106	The summary: both images and tags, "pushed, not deployed".

Both images come from `apps/api/Dockerfile`. The migrate image carries the Prisma command-line tool, which a container serving requests never opens, so it is kept out of the runtime image. Compressed sizes in ECR: 104 MB runtime, 166 MB migrate.

Stage	Lines	What it does
`base`	8–14	Node 22 on Alpine, with OpenSSL (Prisma needs it) and tini, a tiny init program that becomes process 1 so the container exits once Nest has shut down.
`deps`	16–27	Copies only the package manifests and installs with a frozen lockfile, so editing source does not cost a reinstall.
`build`	29–48	Copies the sources, generates the Prisma client, builds the API, extracts a production-only copy with `pnpm deploy` and deletes the Prisma tool, unused database engines and TypeScript from it.
`runtime`	50–57	Copies that folder in, runs as the unprivileged `node` user, exposes port 3000 and starts `node dist/main.js` under tini.
`migrate`	59–68	Copies `prisma/`, installs the Prisma tool 6.19.3 with npm and runs `prisma migrate deploy` by default.

`_deploy-api.yml`

The API half of a release, shared by both environments so they cannot drift. It brings the box's checkout to this commit, pulls an image that already exists in ECR and recreates the containers. It never builds.

Lines	What it does
6–17	Inputs, all required: `environment`, `image_tag` and `reason` (text for the summary).
23–25	Workflow-wide variables: the registry address and `COMPOSE`, the Compose command line reused in the steps.
28–34	The deploy job, named after the environment, 20-minute limit. It declares the caller's environment, which AWS checks. A comment says production's environment carries a required reviewer; today neither does (section 11, gap 3).
37–40	Trades the OIDC token for temporary AWS credentials.
42–50	Finds the box: asks EC2 for a running instance whose `Name` tag equals `vars.API_INSTANCE_NAME`. With none, prints "No running instance tagged ..." and fails.

Lines	What it does
62–82	Sends the deploy script with `aws ssm send-command`, document `AWS-RunShellScript`, comment `deploy <tag>`. The commands are the JSON list on lines 68–80. The step saves the command id.
84–101	Waits and reports. `send-command` succeeds the moment it is queued, which says nothing about the deploy, so the loop asks for the result every 10 seconds, up to 60 times, while the status is Pending, InProgress or Delayed. It copies the output into the summary and fails the job unless the status is Success.

The 13 commands that run on Box A, in order (lines 68–80). They are one script, run by the SSM agent as root.

1. `set -euo pipefail` (68). Stop at the first failing command, treat an unset variable as an error, and count a failure anywhere in a pipeline.
2. `cd /opt/examprep` (69). Where the repository is checked out.
3. `OWNER=$(stat -c %U .git)` (70). Reads which user owns the checkout. SSM runs as root, and git refuses a directory root does not own, so every git command below runs as the owner.
4. `OLD=$(sudo -u $OWNER git rev-parse HEAD)` (71). Remembers the commit the box is on, for command 11.
5. `sudo -u $OWNER git fetch --quiet origin <sha>` (72). Downloads this release's commit. It needs no credentials only because the repository is public (gap 6).
6. `sudo -u $OWNER git merge --quiet --ff-only <sha>` (73). Moves the checkout to this commit, which delivers new `compose.yml`, Caddyfile and Alloy configuration. `--ff-only` refuses, and so stops the release, if the box was edited by hand.
7. `aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin <registry>` (74). Signs Docker in to ECR with the box's own instance role; the password is piped in, never typed.
8. `export API_IMAGE=<registry>/examprep/api MIGRATE_IMAGE=<registry>/examprep/api-migrate API_TAG=<tag>` (75). The variables `compose.yml` substitutes into its image names. They exist only inside this script (gap 1).
9. `$COMPOSE pull` (76). Downloads the two images for that tag.
10. `$COMPOSE up -d` (77). Runs `migrate` to completion, then recreates exam, core and worker. There is no draining, so the API is unreachable while the containers restart. The length of that gap has not been measured.
11. `sudo -u $OWNER git diff --quiet $OLD HEAD -- deploy/Caddyfile deploy/alloy || $COMPOSE restart caddy alloy` (78). `up -d` recreates a container whose Compose entry changed, never one whose mounted config file changed, so Caddy and Alloy restart only when their files moved between the old and new commit.
12. `$COMPOSE ps` (79). Puts container state into the run summary.
13. `./deploy/prune-disk.sh` (80). Deletes old images so the disk does not fill.

The `\$` and `$` in those lines. Three programs read this text in turn: GitHub replaces every `${{ ... }}`; the runner's shell reads the double-quoted string after `--parameters commands=`; and the box's shell runs the result. In a double-quoted shell string a bare `$` is expanded at once, on the runner, and `\$` survives as a literal dollar sign. So `$REGISTRY` and `$COMPOSE` are filled in on the runner, because they are workflow variables the box does not have, and arrive as finished text. `\$OWNER` and `\$OLD`

reach the box as `$OWNER` and `$OLD` and are evaluated there, after earlier lines set them. The `\"` around each command is a literal double quote, because SSM wants a JSON list of strings.

> **A trap when editing lines 68–80.** Writing `$OWNER` without the backslash makes the runner expand it to nothing, and the box receives `sudo -u git fetch ...`. Using `\$COMPOSE` makes the box look for a variable only the runner has.

`_publish-spas.yml`

The front-end half, shared by both environments. `VITE_API_URL` is baked in at compile time, so the sites are built per environment and a staging build can never be promoted: it would call the staging API.

Lines	What it does
5–17	The input `environment`, the OIDC permission, and the Corepack switch.
20–24	The publish job, named after the environment, 20-minute limit, declares the environment for OIDC.
27–40	Check out, install pnpm and Node (version from `.nvmrc`, pnpm cache), `pnpm install --frozen-lockfile`, then trade the OIDC token for temporary AWS credentials.
44–50	Runs `deploy/publish-spas.sh` with four environment variables: `API_ORIGIN`, `SPA_BUCKET`, `STUDENT_DISTRIBUTION`, `ADMIN_DISTRIBUTION`.
52–60	The summary: short commit, API origin, and a reminder that a new service worker waits for old tabs to close.

The comment on line 42 says "three invalidation paths"; the script invalidates four (lines 51–52), and the script is right.

`production.yml`

> **This workflow is disabled.** It has been switched off in the repository's Actions settings since 7 October 2026, because no production box exists and its nightly cron failed every night. `gh workflow enable Production` brings it back. Do not enable it before the design problem below is fixed.

Lines	What it does
12–25	Triggers: a cron at `30 20 * * *`, which is 20:30 UTC and 02:00 IST, and a manual run with a required `reason` and an `image_tag` (default `prod`).
27–33	Same permissions as Staging; group `production-release`.
35–45	Job API calls `_deploy-api.yml` for `production`, with the typed tag or `prod`, and the typed reason or "scheduled nightly release".
47–58	Job SPAs has `needs: api` and no `if`, so it runs only if the API job succeeded.

The design problem: it deploys the moving `prod` tag, which `build-api.yml` pushes separately on a push to `main`. That is the build and deploy race staging had until 7 October. It should build and deploy the sha, as `staging.yml` does (section 12). The header (lines 1–9) also says the `production` environment carries a required reviewer; it has none today.

`scale.yml`

Manual only. It changes how many exam and core containers run and, optionally, the instance size. Do not use it yet: see section 11, gap 1.

Lines	What it does
8–33	Inputs: environment (`staging` or `production`), `exam` and `core` container counts (default 1), `instance_type` (default `keep`) and a required reason.
41–43	Shares the group `<environment>-release` with that environment's release, so a scale and a deploy never recreate containers at once.
64–86	Refuses what cannot work. Each role needs one container. The database allows about 112 connections on a micro, 225 on a small and 450 otherwise, and every container, the single worker included, takes up to 25. The job fails if the wanted total exceeds 90 percent of the ceiling. On the current micro that is 100, which allows three exam and core containers in total plus the worker.
88–95	Finds the box, as `_deploy-api.yml` does.
99–119	When a new size is asked for: `compose down` over SSM so containers get their shutdown grace, a 45-second wait, stop the instance, change its type, start it, then wait until the SSM agent reports Online. The Elastic IP survives.
121–134	Runs `up -d --no-recreate --scale exam=N --scale core=N --scale worker=1`, then `ps`. `--no-recreate` leaves running containers alone. The worker stays at one, and there is no input for it.
136–157	Waits (up to 40 tries) and reports the status, instance type and counts.

5. A staging release, minute by minute

The measured run is 37743849827, on 8 October 2026, commit 29c2750, with both halves changed and a warm build cache. It took 4.9 minutes in total.

Stage	Time	Inside it
What changed	9 s	
Build job	154 s	Cache download about 22 s, compile and package about 33 s, push to ECR about 23 s, cache upload about 29 s, builder teardown 26 s, migrate image 8 s.
API deploy	36 s	Pull, migrate, recreate; 26 s of it is the wait loop.
SPAs	82 s	Node setup 13 s, install 15 s, build and upload 41 s.
Between jobs	about 25 s	Runner start-up, four jobs in a row.

The first run through the pipeline (37623000600, on 7 October, cold cache) took 7.4 minutes, and its build job took 267 seconds. Commit 767aead, not yet pushed, removes the 26 s teardown and narrows the API pattern to `contracts` and `config`.

Kind of push	What runs	Cost
Both halves, warm cache	All four jobs	4.9 minutes, measured.
Both halves, cold cache	All four jobs	7.4 minutes, measured once.
API only	What changed, Build, API	About 3.5 minutes, from adding the measured stages; not measured on its own.
Sites only (a screen change)	What changed, SPAs	About 1.7 minutes once 767aead is pushed; an estimate.
Neither (docs, a workflow file, mobile code)	What changed only	Seconds. The summary says "Nothing to deploy." and the other jobs show as skipped.

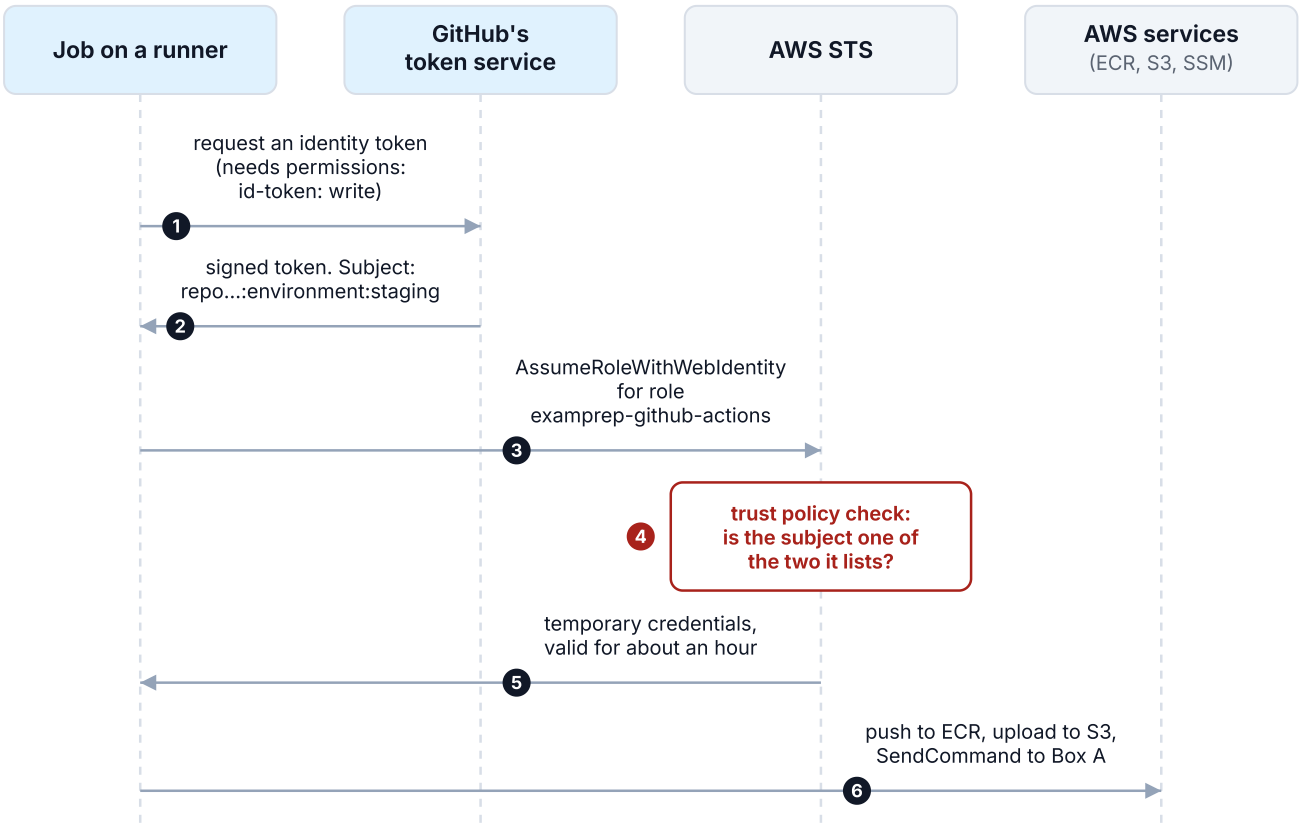
When you push again while a run is going. The group `staging-release` allows one active run. A second push starts a run that waits as pending. If a third push arrives while one run is active and one is pending, GitHub cancels the older pending run and keeps the newest. The cancelled run deployed nothing, which is safe because the newest run measures from the last green run and so covers it. CI is separate: a newer push cancels the older CI run on the same branch. A Scale run joins the same queue.

6. How GitHub is allowed into AWS

No AWS password or access key is stored in GitHub. Each job proves who it is on every run, and AWS hands back credentials that expire.

1. The job declares `environment: staging`, and the workflow grants `id-token: write`.
2. GitHub signs a short-lived token. Its "subject" names the repository and the environment.
3. `aws-actions/configure-aws-credentials` sends the token to AWS STS, naming the role in the repository variable `AWS_ROLE_ARN`.
4. AWS checks the signature against the identity provider registered for `token.actions.githubusercontent.com`, then compares the subject with the role's trust policy.
5. On a match STS returns temporary credentials for the role `examprep-github-actions`, and the rest of the job uses them.

HOW A JOB GETS TEMPORARY AWS CREDENTIALS



No AWS key is stored in GitHub. A job without **environment:** gets a different subject and is refused.

The trust policy. The role accepts exactly two subjects, one per environment. A job must declare `environment: staging` or `environment: production` to get in; a job without one presents a different subject and is refused.

```
repo:Indian-Academy-of-Competitive-Exams@252581941/iace-exam-
portal@1340318821:environment:staging
repo:Indian-Academy-of-Competitive-Exams@252581941/iace-exam-
portal@1340318821:environment:production
```

> **The spelling trap.** This subject is GitHub's immutable form, with the numeric ids of the organization and the repository after an `@`. The plain `repo:org/repo:environment:staging` form every tutorial shows is refused. A subject that does not match gives "Not authorized to perform sts:AssumeRoleWithWebIdentity", which is what failed every AWS job from 4 to 7 October 2026.

To see the subject form GitHub issues for this repository, ask its API:

```
gh api repos/Indian-Academy-of-Competitive-Exams/iace-exam-
portal/actions/oidc/customization/sub
```

What the role may do. The role's inline policy has nine statements.

Statement	Allows	On
PushImages	Push and read image layers	The two ECR repositories
EcrLogin	`ecr:GetAuthorizationToken`	All resources
PublishTheSpas	Put, get, delete and list objects	The bucket `examprep-staging-spas`
InvalidateTheEdge	`cloudfront:CreateInvalidation` and `GetInvalidation`	All resources
RunShellDocument	`ssm:SendCommand`	The `AWS-RunShellScript` document
DeployToBoxA	`ssm:SendCommand`	EC2 instances tagged `Name=examprep-api-staging`
ReadTheResult	`ssm:GetCommandInvocation`, `ListCommandInvocations`, `DescribeInstanceInformation`	All resources
ReadTheShape	`ec2:DescribeInstances`, `DescribeInstanceStatus`, `rds:DescribeDBInstances`	All resources
ResizeBoxA	EC2 stop, start and `ModifyInstanceAttribute`	EC2 instances tagged `Name=examprep-api-staging`

`ssm:SendCommand` is split in two because a command needs permission on two things, the document and the instance. A single statement that put the instance-tag condition on the document as well failed on 7 October, because a document carries no such tag.

The box's own identity. Box A runs with the instance profile `examprep-box`, and Box B uses the same one. It carries `AmazonSSMManagedInstanceCore`, which lets SSM reach the box (there is no SSH and no inbound port 22); `CloudWatchAgentServerPolicy`; and an inline policy for the media bucket, `ssm:GetParameter` on `/examprep/*`, `kms:Decrypt` and ECR pull. The box signs Docker in to ECR with this profile (command 7).

Where configuration lives. GitHub holds no secrets for this pipeline; everything is a variable.

Scope	Variable	Value or use
Repository	`AWS_ROLE_ARN`	The ARN of `examprep-github-actions`.

Scope	Variable	Value or use
Environment `staging`	`API_INSTANCE_NAME`	`examprep-api-staging`, the tag on Box A.
Environment `staging`	`API_ORIGIN`	`https://api.staging.examprep.iace.co.in`, compiled into the sites.
Environment `staging`	`SPA_BUCKET`	`examprep-staging-spas`.
Environment `staging`	`STUDENT_DISTRIBUTION`	CloudFront id `EFX7ZP0HCI3C3`.
Environment `staging`	`ADMIN_DISTRIBUTION`	CloudFront id `E2QYUOD6YASE4V`.
Environment `staging`	`DB_INSTANCE_ID`	`examprep`, read by `scale.yml`.
Environment `production`	none yet	Production needs the same six (section 12).

7. The deploy folder, file by file

`deploy/` holds everything that runs on, or is shipped to, the two boxes. Most of it is Docker Compose, so its syntax comes first.

Syntax	Meaning
`${VAR:-default}`	The value of `VAR`, or `default` when `VAR` is unset or empty. `${VAR:?message}` instead refuses to start when `VAR` is unset.
`&name` and `*name`	A YAML anchor labels a block and an alias copies it elsewhere, so a setting is written once.
`depends_on` with `condition: service_completed_successfully`	The service starts only after the named one has run to completion and exited with code 0. Without a condition it waits only for the other to have started.
`cpu_shares`	A CPU weight, not a limit. It decides who yields when the box is busy, and an idle box can be used whole.
`mem_limit`	A hard cap. A container that exceeds it is killed.
Named volume	Storage Docker manages, which outlives the container (`caddy-data:/data`).
Bind mount	A host file or folder placed inside the container (`./Caddyfile:/etc/caddy/Caddyfile:ro`, read-only).
`ports` and `expose`	`ports` publishes a port on the host; `expose` only documents one on the internal network.

`deploy/compose.yml`

Box A: six containers from one file. Postgres is RDS and Valkey is Box B, so nothing else runs here. Only Caddy publishes a port, so the box's firewall has two holes in it.

Lines	What it does
9	`name: iace-api` is the project name; Compose prefixes container and network names with it.
24–28	`x-logging: &logging` defines the log settings once (Compose ignores top-level keys starting with `x-`, so one can hold an anchor): the `json-file` driver keeps at most three files of 10 MB per container. Every service refers to it as `*logging`.
32–44	Service `migrate`. Its image is `${MIGRATE_IMAGE:-iace-migrate}:${API_TAG:-local}`: the ECR image when the deploy exports the variables, a local name otherwise. The `build` block (context `..` is the repository root, target `migrate`) lets `up -d --build` work with no registry. `restart: 'no'` lets it exit when done.
47–53	Service `exam`. `&api-tag` names the image string `${API_IMAGE:-iace-api}:${API_TAG:-local}` and `&api-image` names the build block, so `core` and `worker` reuse both. `restart: unless-stopped` brings it back after a crash or reboot.
54	`env_file: .env` gives the container every key in `deploy/.env` (section 8).

Lines	What it does
55–61	`API_ROLE: exam` chooses what this container answers. `UV_THREADPOOL_SIZE` sizes Node's I/O threads, `NODE_OPTIONS` caps the JavaScript heap at `${EXAM_HEAP_MB:-384}` MB, the CPU weight is `${EXAM_CPU_SHARES:-2048}` and the memory cap is `${EXAM_MEM:-1g}`.
63–66	30 seconds to shut down, so BullMQ can close and Prisma can drain (the default 10 is too short); and a wait for `migrate` to finish with code 0, so a failed migration keeps the API containers from starting.
68–73	Health check, anchored as `&api-health`: `wget --spider -q` on `http://127.0.0.1:3000/health/ready` every 10 s, 5 s timeout, 10 retries, 20 s of grace. `/health` answers 200 even when degraded; only `/health/ready` fails when a dependency is down.
75–92	Service `core` reuses `*api-tag`, `*api-image` and `*api-health`. Role `core`, thread pool 8, weight 1536.
94–112	Service `worker`: exactly one, always. Role `worker`, weight 1024. Its queue policy relies on running in one process; two workers would put two sweeps over the same keys.
115–144	Service `alloy`, the log and metrics shipper, image `grafana/alloy:v1.12.0`, weight 256 (the first to yield). `command` starts it with its data path and `/etc/alloy/config.alloy`. `environment` passes `ENVIRONMENT`, the `LOKI_` and `PROM_` settings and `METRICS_TOKEN`. Volumes: the config file read-only, Docker's socket read-only (which still makes Alloy root-equivalent on the box) and `alloy-data`, where it records how far it has read each log. `expose: '12345'` keeps its UI off the host.
146–164	Service `caddy`, image `caddy:2-alpine`, the only service with `ports` (80 and 443). It receives `ACME_EMAIL` and `API_HOST`. Volumes: the Caddyfile read-only, `caddy-data` (the issued certificates; losing it makes every restart ask Let's Encrypt again, which allows only five duplicate certificates a week per hostname) and `caddy-config`. It starts after `exam` and `core` have started, not after they are healthy.
166–169	Declares the three named volumes.

CPU is a weight, never a quota. The weights 2048, 1536 and 1024, with Caddy's default 1024 and Alloy's 256 in the total, give a floor under full contention on 2 vCPU of 0.70 CPU for exam, 0.52 for core and 0.35 for the worker. With core and the shipper idle, which is what a live sitting looks like, exam can take the whole box. Memory stays a hard cap. Line 22 says the disk is 30 GB; `deploy/prune-disk.sh` and the README say 20 GB.

THE ORDER COMPOSE STARTS THE CONTAINERS ON UP -D

First: nothing to wait for

migrate → Runs the database migrations, then exits

alloy → Has no dependencies, so it starts beside migrate

Second: after migrate has exited with code 0

exam → Answers the sitting paths

core → Answers everything else

worker → Scores submitted attempts; exactly one

Third: after exam and core have started

caddy → Takes ports 80 and 443 and sends requests to exam and core

`deploy/Caddyfile`

Caddy is the web server in front of the API. It terminates HTTPS, fetches its certificate from Let's Encrypt and sends each request to the exam or the core containers.

Lines	What it does
6–10	Global options: `{$ACME_EMAIL}` reads an environment variable Compose passes in and registers the Let's Encrypt account. The site block for `{$API_HOST}`: naming a host makes Caddy obtain and renew a certificate for it.
11	`@exam` is a named matcher: seven path patterns, six for starting, reading, saving and submitting a sitting and one for the leaderboard. `*` stands for one path segment.
13–22	Requests matching `@exam` go to the exam containers. `dynamic a` asks Docker's DNS for every container of the service `exam` on port 3000 and asks again every 10 seconds, so scaling needs no edit. `max_fails 3` within `fail_duration 30s` takes a failing container out for 30 seconds; the check is passive.
24–32	Everything else goes to `core`, set up the same way.
36–39	The access log goes to standard output as JSON, where Docker captures it and Alloy ships it.

`deploy/alloy/config.alloy`

Alloy is Grafana's agent. It sends every container's logs to Loki and the API's metrics to Prometheus, both in Grafana Cloud, and nothing in the API knows it exists. The file is a graph of components: each block is one stage, `forward_to` names the stage that receives its output, and `sys.env("X")` reads an environment variable Compose passed in.

Lines	What it does
11–13	`discovery.docker "boxa"` asks Docker's socket for the running containers.
16–30	`discovery.relabel "containers"`: label each log stream `service_name` with its Compose service (`exam`, not `/iace-api-exam-1`), and drop Alloy's own container so one shipping error cannot become a loop.
32–38	`loki.source.docker` tails the discovered containers, applies those rules, adds the label `env` from `ENVIRONMENT` and passes the lines on.
41–54	`loki.process "levels"` parses each JSON line and promotes its `level` field to a label. Only `level` and `service_name` are labels, because each label is an index; per-request detail stays in the line.
56–65	`loki.write "cloud"` sends the lines to `LOKI_URL` with `LOKI_USERNAME` and `LOKI_PASSWORD`.
73–102	`discovery.relabel "api"` picks the metrics targets: keep only `exam`, `core` and `worker`; build each address from the container's network address plus port 3000, because nothing publishes 3000; label the Compose service `role` and the container name `instance`, so four replicas are four series.
104–111	`prometheus.scrape "api"` scrapes `/metrics` on those targets every 15 seconds, with `METRICS_TOKEN` as the bearer token.
113–126	`prometheus.remote_write "cloud"` pushes the samples to `PROM_URL` with `PROM_USERNAME` and `PROM_PASSWORD`, adding the label `env`.

`deploy/prune-disk.sh`

Keeps Box A inside its 20 GB. What fills the disk is Docker images and build cache, not logs. The deploy runs it as its last step (command 13), and it is safe at any time because Docker refuses to remove an image a running container uses.

Lines	What it does
19–27	`set -euo pipefail`; `KEEP` defaults to 2, so the newest two builds stay; `say` prints a heading; shows disk use before the cleanup.
31–32	`docker container prune -f` removes stopped containers first, so the images behind them stop counting as in use. `migrate` is always among them.
36–49	Finds our own images by name (`examprep/api`, `examprep/api-migrate`, `iace-api`, `iace-migrate`), lists each repository's image ids newest first without duplicates, and removes every id past the first `KEEP` with `docker rmi -f`. Caddy, Alloy and Valkey images are left alone.
51–59	Removes dangling image layers, build cache (the big one on a box that builds) and unused networks.
61–63	Shows disk use afterwards.

> **Never add `--volumes` to a prune.** The volumes hold Caddy's issued certificates and, on Box B, Valkey's append-only file. Let's Encrypt allows five certificates per hostname per week, so one careless prune can lock the domain out for seven days (lines 16–18).

`deploy/publish-spas.sh`

Builds both sites and publishes them. `_publish-spas.yml` runs it with four arguments: the API origin, the bucket and the two CloudFront ids. An environment is a build, not a variable, because `VITE_API_URL` is substituted at compile time.

Lines	What it does
8–18	`set -euo pipefail`; four positional arguments, where `${1:?usage…}` aborts with a usage message when one is missing; moves to the repository root and records the current commit as `HEAD`.
21–29	`deployed()` reads `version.json` from S3, not from the edge, which may still hold the old copy. If both sites already serve this commit the script prints "both sites already serving … nothing to publish" and exits, unless `FORCE=1`.
31	Builds `@iace/exams` and `@iace/admin` with `VITE_API_URL` set.
33–53	`publish()` for one site. It syncs `assets/` with a one-year `immutable` cache header, because hashed files never change; writes `version.json` (commit, build time, API origin); syncs the rest with `no-cache`; and invalidates four paths: `/index.html`, `/sw.js`, `/manifest.webmanifest` and `/version.json`.
55–58	Publishes `apps/exams/dist` under `student` and `apps/admin/dist` under `admin`, then prints the short commit.

Assets go first and the shell second; the other order serves a shell pointing at chunks not yet uploaded. Old assets are never deleted: a tab opened before the deploy still loads a chunk by its old name, and keeping them costs about a cent a year. The invalidation names four paths and never `/*`, which would throw away the whole asset cache for nothing.

`deploy/valkey/compose.yml` and `deploy/valkey/.env.example`

Box B runs Valkey, which holds every live sitting, every session and every sign-in code. It runs one process per environment, because `maxmemory` is per process and a full instance under `noeviction` refuses writes: a staging runaway must not stop production saving a candidate's answers.

Lines	What it does
15–20	Project name `iace-valkey`; service `valkey-prod`, image `valkey/valkey:8-alpine`, restarted unless stopped. Defined, not started.
21–33	The server's flags. `--appendonly yes` logs every write so a restart keeps the data. `--maxmemory-policy noeviction` because BullMQ requires it. `--maxmemory` is `${PROD_MAXMEMORY:-1200mb}` and is always set, or Valkey grows until the kernel kills it. `--requirepass` takes `${VALKEY_PROD_PASSWORD:?…}`, so Compose refuses to start without a password.
35–40	`REDISCLI_AUTH` lets the health check's `valkey-cli` authenticate without the password on its command line. `'${PRIVATE_IP}:6379:6379'` publishes the port only on the VPC address. The named volume `prod-data` keeps the append-only file in `/data`.
42–47	60 seconds to shut down, so the last second of writes is flushed. Health check `valkey-cli ping` every 10 seconds, anchored as `&ping`.
49–70	`valkey-staging`: the same, with its own password, `${STAGING_MAXMEMORY:-256mb}`, host port 6380 mapped to container port 6379, volume `staging-data` and `*ping`.
72–74	Declares the two named volumes.

The three locks on this box are the security group, the bind to `PRIVATE_IP` and `requirepass`; the last still holds after someone edits the group wrong. In `valkey/.env.example`, line 8 is `PRIVATE_IP` (the example is `10.0.1.20`), lines 11 and 12 are the two passwords (line 3 gives the command to generate them, `openssl rand -base64 36`), and lines 16 and 17 are the two memory ceilings. Compose reads the file from beside `compose.yml` with no flag, and a copy goes to SSM as `/examprep/valkey/env`.

`deploy/.env.example`

The template for `deploy/.env` (section 8 explains the file). Compose reads it to fill placeholders and the API containers read it as their environment.

Lines	What it does
13–14	`API_HOST` and `ACME_EMAIL`: the one hostname Caddy needs a certificate for, and the Let's Encrypt contact. Let's Encrypt refuses `*.amazonaws.com`, so an EC2 hostname cannot be used.
25–33	`CPU_SHARES`, `MEM` and `HEAP_MB` for each of `EXAM_`, `CORE_` and `WORKER_`. Heap is in megabytes, about 80 percent of the memory limit. Production overrides the memory (lines 17–18).
45–46	`NODE_ENV` and `OTP_SENDER`. Staging runs `development` and `console`; production needs `production` and `sms`.
53–59	`DATABASE_URL` for RDS, with `connection_limit=25` and `pool_timeout=20`; `REDIS_URL`, Box B's private address and the environment's port (6380 for staging, 6379 for production), with the password after an empty username.
62–65	`JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, `PIN_PEPPER` and `METRICS_TOKEN`. The file's header (line 5) gives the command to generate each: `openssl rand -base64 48`.

Lines	What it does
70–82	`S3_REGION`, `S3_BUCKET`, the key pair and `S3_FORCE_PATH_STYLE` (on AWS the key pair stays empty and the instance profile signs requests), and `MEDIA_BASE_URL`, the CloudFront domain for question images, with no bucket name in the path.
85–88	`CORS_ORIGINS`, the browsers allowed to call the API (the two site domains), and `TRUST_PROXY_HOPS=1`, because Caddy is the one hop in front.
93–106	`ENVIRONMENT`, which becomes the `env` label on every log line and metric; `LOKI_URL`, `LOKI_USERNAME`, `LOKI_PASSWORD` and the three `PROM_` equivalents, read by Alloy. The two usernames differ.

`deploy/README.md`

The by-hand guide. It was written before the pipeline existed, so parts read as if the box builds its own images (lines 85 and 175). Its count of three boxes on line 3 includes production's API box, which does not exist yet.

Lines	What it does
13–65	Before anything (DNS, security groups with no inbound SSH, SSM Session Manager, IMDSv2, 4 GB to build), then the env file: create, restrict, push to SSM, pull on a new box, the 4 KB limit.
67–132	Box B's Valkey, Box A's first start, seeding the first super admin by SQL, and telemetry.
134–184	Publishing the sites by hand, keeping the disk inside 20 GB (including a weekly cron as a backstop, lines 157–159) and redeploying by hand.
186–239	The Docker plugins Amazon Linux lacks, the `pg_trgm` extension to create once as the master user, troubleshooting, and what was verified on 26 September 2026.

8. The environment file

`deploy/.env` is one plain-text file of `KEY=value` lines per environment. It lives at `/opt/examprep/deploy/.env` on the box, it is git-ignored, the pipeline never writes it, and it is the only place a credential lives on the box. The developer's local copies sit outside the repository in `~/.examprep/` and are linked into `deploy/` as `.env.staging`, `.env.production` and `.env.local`, also git-ignored. This section names keys only.

Compose reads the file in two ways.

1. `--env-file deploy/.env`, inside the `COMPOSE` variable. Compose uses it to fill the `${…}` placeholders in `compose.yml` itself. A variable already set in the shell wins over the file, which is how the deploy script supplies `API_IMAGE`, `MIGRATE_IMAGE` and `API_TAG`, none of which is in the file.
2. `env_file: .env` on `exam`, `core` and `worker` (lines 54, 80 and 101). Every key in the file becomes an environment variable of those three containers, so the API also receives the keys meant for Alloy and Caddy, which it does not read.

Read by	Keys
Compose, for placeholders	The `EXAM_`, `CORE_` and `WORKER_` sizing keys.
Caddy, through `environment:`	`API_HOST`, `ACME_EMAIL`.
Alloy, through `environment:`	`ENVIRONMENT`, `LOKI_URL`, `LOKI_USERNAME`, `LOKI_PASSWORD`, `PROM_URL`, `PROM_USERNAME`, `PROM_PASSWORD`, `METRICS_TOKEN`.
The migrate container	`DATABASE_URL` only.
The API (exam, core, worker)	`NODE_ENV`, `OTP_SENDER`, `DATABASE_URL`, `REDIS_URL`, the secrets, the `S3_` keys, `MEDIA_BASE_URL`, `CORS_ORIGINS`, `TRUST_PROXY_HOPS`, the mail and SMS keys and `METRICS_TOKEN`.

The SSM master copy. The parameter `/examprep/staging/env` (a SecureString) holds a copy of the file, and `/examprep/valkey/env` holds Box B's. Nothing reads them at runtime: the API validates its own environment and boots whether or not AWS answers. A replacement box pulls the copy. The instance profile may read parameters and may not write them, so a box cannot poison what its replacement pulls; push from a machine with admin credentials, not from the box, and push after every change:

```
aws ssm put-parameter --name /examprep/staging/env --type SecureString --overwrite --value
file://deploy/.env
```

On a new box, one command brings the file back:

```
aws ssm get-parameter --name /examprep/staging/env --with-decryption --query Parameter.Value
--output text > deploy/.env
```

Problems in the staging file today.

- Eight keys use names the API no longer reads: `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_FROM`, `MSG91_AUTH_KEY`, `MSG91_SENDER_ID` and `MSG91_OTP_TEMPLATE_ID`. The current names are `MAIL_USER`, `MAIL_PASSWORD`, `SMS_PROVIDER_URL`, `SMS_PROVIDER_KEY`, `SMS_SENDER_ID` and `SMS_TEMPLATE_OTP`.
- Ten helper keys are read by nothing: `AWS_REGION`, `BOX_A`, `BOX_B`, `DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `VALKEY_HOST`, `VALKEY_PORT` and `VALKEY_PASSWORD`.
- `MEDIA_BASE_URL` appears twice.
- The SSM copy, written on 4 October, is older than the local file, written on 7 October. The local file is 6,065 bytes and the Standard tier of Parameter Store holds 4 KB, so it must be stripped of comments or stored in the Advanced tier ($0.05 a month).
- `PIN_PEPPER` in `deploy/.env.example` (line 64) is dead: the API has no such setting.
- Line 92 of `.env.example` says `NODE_ENV` is `production` on both environments; staging runs `development`.

9. When something fails

On the box, `compose` below means `docker compose -f deploy/compose.yml --env-file deploy/.env`, run in `/opt/examprep`. Open a shell with `aws ssm start-session --target <instance-id>`; there is no SSH. Use `compose` for `logs` and `ps`, and do not run `compose up -d` by hand until gap 1 in section 11 is fixed.

What you see	What it means	What to do
"Could not assume role with OIDC: Not authorized to perform sts:AssumeRoleWithWebIdentity"	The job's subject is not in the role's trust policy: the job has no `environment:`, or the policy has the wrong spelling.	Compare with the output of `gh api repos/<org>/<repo>/actions/oidc/customization/sub`.
"not authorized to perform: ssm:SendCommand"	The role's policy does not cover the document or the instance tag.	Check the statements RunShellDocument and DeployToBoxA.
"No running instance tagged examprep-api-staging"	Box A is stopped, terminated or renamed.	Start it, or fix the `API_INSTANCE_NAME` variable.
SSM status Failed, output mentions "Not possible to fast-forward" or local changes	The box's checkout was edited by hand, or `staging` was force-pushed.	Open a session, inspect `git status` as the checkout's owner, resolve, rerun.
"manifest unknown" or "pull access denied" during pull	That image tag is not in ECR: a mistyped tag on a manual run, or the lifecycle rule expired it.	Use a tag that exists, or push again to rebuild.
`migrate` exits non-zero and exam, core and worker never start	A migration failed, and the API containers wait for it to succeed.	Read `compose logs migrate`. The old containers were already stopped, so this is an outage until it is fixed.

What you see	What it means	What to do
Containers unhealthy	The API refused to boot and named the variable in its log.	Read `compose logs exam`.
SPAs job skipped	The build or API job failed or was cancelled; publishing is withheld on purpose.	Fix the API half and rerun.
"Nothing to deploy."	The push touched neither half.	Nothing.
A run shows cancelled while it was pending	A newer push replaced it in the queue.	Nothing: the newer run measures from the last green run and covers it.
"both sites already serving … nothing to publish"	`publish-spas.sh` found the same commit in `version.json`.	Set `FORCE=1` only if you must republish.
Deployed, but students still see the old site	The service worker takes over only when every old tab is closed.	Expected.

What broke on the way.

- From 4 to 7 October 2026 every job that needed AWS failed with "Could not assume role with OIDC", because the trust policy's subject did not match the one GitHub issues. Fixed on 7 October.
- On 7 October the deploy then failed with "not authorized to perform: ssm:SendCommand on resource … document/AWS-RunShellScript". One IAM statement put the instance-tag condition on the document as well, and a document has no such tag. It was split into RunShellDocument and DeployToBoxA.
- Until 7 October the image build was a separate workflow that started beside the deploy, so the deploy pulled the previous push's image and then published the new sites over it. Commit 292577d made the build a job inside the Staging run and deploys the sha.
- The deploy never updated the box's checkout, so changes to `compose.yml`, the Caddyfile and the Alloy configuration never reached the box. The same commit fixed it.
- The nightly Production cron failed every night because no production box exists. The workflow was disabled on 7 October.
- The first fully green release was on 7 October 2026: run 37623000600, commit 292577d.

10. Everyday operations

Release to staging. Push the branch.

```
git push origin staging
```

Watch it.

```
gh run watch
```

```
gh run list --workflow staging.yml
```

See what is live. Each site publishes the commit it was built from, and the API has a health route.

```
curl https://staging.examprep.iace.co.in/version.json
```

```
curl https://admin.staging.examprep.iace.co.in/version.json
```

```
curl https://api.staging.examprep.iace.co.in/health
```

Redeploy without a new commit. In GitHub, open Actions, then Staging, then **Run workflow**, and type the tag (default `staging`, the last image built). It treats both halves as changed, builds nothing, deploys that tag, then publishes the sites, which does nothing if they already serve this commit. From the command line:

```
gh workflow run staging.yml --ref staging -f image_tag=staging
```

Roll back the API. Run the same workflow with the full sha of an earlier commit as the image tag. The image must still be in ECR; this lists what is there:

```
aws ecr describe-images --repository-name examprep/api --region ap-south-1 --query
'sort_by(imageDetails,&imagePushedAt)[].[imageTags,imagePushedAt]'
```

```
gh workflow run staging.yml --ref staging -f image_tag=<full-sha>
```

> **A rollback undoes the API image and nothing else.** The database schema stays where the newer migrations left it, and the two sites stay on the newer build. To take the sites back, revert the commit and push. The box's checkout also moves to the commit the run started from, which on a manual run is the current tip of `staging`, so `compose.yml` and the configuration files are the newest ones whatever tag you type.

Scale. Actions, then Scale. Read gap 1 in section 11 first.

By hand, when GitHub or ECR is unavailable. Open a session on the box and build there, as `deploy/README.md` describes (lines 172–184).

```
git pull && docker compose -f deploy/compose.yml --env-file deploy/.env up -d --build
```

11. Known gaps today

These are the state on 8 October 2026.

1. **The deployed image tag is not recorded on the box.** `API_IMAGE`, `MIGRATE_IMAGE` and `API_TAG` exist only inside the deploy's shell. Any later Compose command without them, whether the Scale workflow or someone running `up -d` by hand, resolves the image to `iace-api:local`, a stale locally built image, or tries to build on the box. Avoid Scale until this is fixed.
2. **Rollback depth is about three releases, not ten.** Each push stores three entries per repository (the tagged image and two untagged ones from the build's provenance attestation) and the lifecycle rule counts all of them. Box A keeps only the newest two builds locally.
3. **The `staging` and `production` environments have no protection.** There is no required reviewer and no branch restriction, so anyone with write access can push a branch whose workflow declares `environment: staging` and receive the deploy role, which can run commands as root on Box A. There are no branch rulesets on `main` or `staging` either. Only the repository admin, `suryaprakash-sp`, can change these.
4. **CI does not gate the staging deploy.** A push that fails its tests still deploys.
5. **The repository is public, and staging runs with `NODE_ENV=development` and `OTP_SENDER=console`,** which returns the sign-in code in the API response. Anyone can sign in to staging, so keep real data out of it.
6. **Box A's `git fetch` needs no credentials only because the repository is public.** If it goes private, the box needs a read-only deploy key.
7. **A release replaces the API containers with a gap.** There is no load balancer to drain through, so requests that arrive while the containers restart fail. The length of the gap has not been measured.
8. **The staging env file has dead and misnamed keys and a stale SSM copy** (section 8). Settings someone believes are applied may be ignored.
9. **Three comments are stale:** `deploy/compose.yml` line 22 says the disk is 30 GB (the prune script says 20), `deploy/.env.example` line 92 says `NODE_ENV` is `production` on both environments, and `_publish-spas.yml` line 42 says three invalidation paths (the script has four).
10. **There are no CloudWatch alarms.** One $100 monthly budget exists, so a stopped box, a full disk or a failed container is found by looking.
11. **The box itself was not inspected.** What this document says about its contents comes from the files and the pipeline's own output.

12. What production needs

No production infrastructure exists yet. What does exist is the workflow files, a `production` GitHub environment with no variables and no protection, a trust-policy subject for it, and a `valkey-prod` service that is defined but not started.

Before the first production release:

- Its own Box A (t4g.large, tagged for production), Elastic IP and DNS record; its own SPA bucket and three CloudFront distributions; a production database on the RDS instance; and `valkey-prod` started on Box B, which must first be resized (it is a t4g.micro with 1 GB, and production's memory ceiling alone is 1200 MB).
- A separate instance role for production. Today both boxes share `examprep-box`, which can read every parameter under `/examprep/*`, so the staging box could read production's secrets.
- A separate deploy role trusted only by `environment:production`, plus a required reviewer and `main` as the only branch allowed to use that environment.
- The six environment variables that `staging` already has, set on `production`.
- A production env file: `NODE_ENV=production`, `OTP_SENDER=sms`, `SMS_PROVIDER_URL`, `MAIL_USER` and `MAIL_PASSWORD` (the API refuses to start without these three in that mode: `apps/api/src/common/messaging/messaging.module.ts` lines 31–38), `SMS_PROVIDER_KEY` (not checked at start, but sent as the provider's authorization header), Valkey port 6379, the larger memory and heap values, production's bucket and origins, and `ENVIRONMENT=production`. It is pushed to `/examprep/production/env`.
- Fix the nightly design before re-enabling the workflow. `production.yml` deploys the moving `prod` tag, which has the build and deploy race staging had. It should build and deploy the sha, as staging does.
- Database: create the `pg_trgm` extension as the master user, revoke public CONNECT on both databases, and insert the first super admin by SQL.
- `examprep.iace.co.in` is currently a CNAME to the old platform (ThinkExam). The cutover is a scheduled switch, not a new record.

Soon after:

- Rollback depth (gap 2).
- Alarms for box status, disk, RDS connections and the CPU-credit surcharge, and a Grafana alert when a container's `up` metric is 0.
- Migrations that work with the previous image, because rolling back an image does not roll back the schema.
- A deploy key, if the repository goes private.

Public or private. The pipeline depends on GitHub environments and on the free ARM runner, so the repository's visibility matters.

Feature	Public and Free (today)	Private and Free	Private and Team (about $28 a month at 7 seats)
Actions minutes	Unlimited	2,000	3,000

Feature	Public and Free (today)	Private and Free	Private and Team (about $28 a month at 7 seats)
ARM runner	4 vCPU, free	2 vCPU, uses minutes	2 vCPU, uses minutes
Environments (variables, the OIDC subject)	Yes	No	Yes
Required reviewer on production	Yes	No	No (Enterprise only)
Branch protection	Yes	No	Yes

CI uses about 15 billable minutes a run and ran 27 times in 30 days, about 400 minutes. Going private on the Free plan breaks the current design, because both the workflows and the trust policy depend on environments.

CodeBuild. Moving to AWS CodeBuild is not recommended. It would mean rewriting seven workflows and obtaining an admin-approved GitHub connection, to replace a pipeline whose faults were configuration, not the platform.